

CMCMIS_SIMPLIFIED

Phase 9 - Job Card Detail + Observations + Engineer

FULLY SEALED · SEAL-PACKED · SENIOR ENGINEERING DOCUMENTATION

Generated for DS-style CMCMIS production documentation. Content source: Phase 9 implementation transcript and sealed project context.

LOCKED IDENTITY

This document records Phase 9 as the Job Card operational layer: detail page, observations, engineer execution, task checklist, documents, completion gates, closure, reopen, JR draft edit/cancel, RBAC, smoke verification, and future handoff logic.

IMPORTANT FILE-NAME NOTE

The requested output filename says phase7SLICE2twofullySEALEDsealpacked.docx. The content inside this document is intentionally Phase 9. A second accurately named copy is also produced for safety.

Sealed Snapshot

Field	Value
Project	CMCMIS_SIMPLIFIED - Computerized Maintenance & Calibration Management Information System
Phase	Phase 9
Module	Job Card Detail + Observations + Engineer Execution
Delivery State	END-TO-END COMPLETED / SEALED
Style	DS-style senior technical documentation with colored blocks and step-by-step logic
Code Policy	No heavy source-code dumping; only small logic snippets where useful

Table of Contents - DS Navigation Map

- 01. 1. Mission Briefing
- 02. 2. Sealed Context
- 03. 3. What Phase 9 Delivered
- 04. 4. Database Design Logic
- 05. 5. Migrations 300-304
- 06. 6. Job Card State Machine
- 07. 7. Backend Architecture
- 08. 8. Frontend Architecture
- 09. 9. Thirteen Tab UX
- 10. 10. Engineer Execution Flow
- 11. 11. Mark Complete Gates
- 12. 12. Closure and Reopen
- 13. 13. Documents and File Storage
- 14. 14. Task Checklist
- 15. 15. JR Loose Ends
- 16. 16. RBAC and Row-Level Scope
- 17. 17. Smoke Verification
- 18. 18. Honest Scope Notes
- 19. 19. How to Think Like Backend Engineer
- 20. 20. Next Phases

1. Mission Briefing - What Phase 9 Means

WHAT WE DID

Phase 9 turns Job Cards from list rows into a full operational execution workspace. Lab Engineers can open a card, start work, save observations, manage tasks, upload documents, mark completion, and pass the card to Lab In-Charge for closure. Lab In-Charge/Super Admin can verify-close or reopen with reason.

SENIOR ENGINEERING MEANING

This is the point where the CMCMIS workflow becomes a real production maintenance and calibration system: request -> approval -> job card -> engineer work -> evidence -> completion -> closure -> audit trail.

Phase 9 Output Snapshot

Field	Value
DB	5 migrations, 53 new nullable columns, 6 new tables, 45 seeded task-library rows, 3 JR cancel columns.
Backend	New Job Card state machine, 17 endpoints, tab patching, documents via multer, task checklist, completion gates, closure and reopen.
Frontend	13-tab detail page, 6 hooks, schemas, autosave, task checklist, documents, mark-complete gate panel, closure and reopen UI.
Smoke	30/30 passed including A1..A29 plus setup; three cross-engineer checks skipped due to single-engineer DB.
Scope deviation	Maintenance Details and Spares Used multi-row tabs are prepared as Slice 2 stubs; data workarounds exist through Observations/Contract-Warranty.

1.1 End-to-end user journey

```
JC List -> click J00024218 -> Job Card Detail
-> Header shows code, status, equipment summary, assigned engineer
-> Engineer clicks Start Work
-> Engineer fills 13 tabs, observes auto-save and manual save
-> Engineer manages task checklist and documents
-> Mark Complete validates four gates
-> Lab In-Charge verifies closure with customer acknowledgement
-> If issue found, LIC/SA reopens with reason
-> History + audit remain intact
```

1.2 Phase 9 in one sentence

ONE-LINE LOCK

Phase 9 is the Job Card execution cockpit for the engineer and the verification cockpit for the Lab In-Charge.

2. Sealed Context - What This Phase Inherits

Layer	State	What Phase 9 inherits
Phase 0-5	Already sealed	Foundation, domain, architecture, database, auth/RBAC, equipment backbone.
Phase 6	Sealed	Job Request/Job Card initial backbone: list, draft, submit, state-machine baseline.
Phase 7 Slice 2	Sealed	Job Request detail, approve/reject, convert to Job Card, auto-create + assign.
Phase 8	Sealed	Dashboard + Inquiry module with cache-aware KPI layer.
Phase 9	This document	Job Card detail, engineer execution, observations, documents, closure, reopen, JR draft edit/cancel.

SEALED MEANS ADDITIVE

Phase 9 follows the same doctrine as previous locked phases: add columns, add tables, add routes, add modules, and add indexes; do not drop, rename, repurpose, or make destructive schema changes.

2.1 Critical inherited doctrines

- **Single choke-point state machines:** Status changes must flow through one entity-specific transition function.
- **Audit pairing:** Every state transition writes an audit row and a status-history row in the same transaction.
- **RBAC is layered:** FE hides actions, routes authorize permissions, service re-checks ownership, state machine verifies transition legality.
- **Legacy database respect:** Old CMMS data remains readable and valid; MVP features are additive.
- **employee_id actor identity:** Actor fields use employee_id VARCHAR(7), matching audit, JR, and legacy employee master patterns.

2.2 Why Phase 9 depends on Phase 7 Slice 2

Phase 7 Slice 2 created the bridge from a submitted Job Request to an assigned Job Card. Phase 9 consumes those assigned Job Cards and executes the engineer-side lifecycle. Without the auto-created JC and assigned engineer from Phase 7 Slice 2, Phase 9 would have no operational work queue.

Phase 7 Slice 2: SUBMITTED JR -> APPROVE + ASSIGN -> Job Card created
 Phase 9: ASSIGNED JC -> IN_PROGRESS -> COMPLETED -> VERIFIED_CLOSED

3. What Phase 9 Delivered

3.1 Delivered scope by layer

Layer	Delivered items
Database	Migrations 300-304, 53 new fields on Job Card, child tables, task library, status history, JR cancel metadata.
Backend - Job Cards	State machine, detail/history, patch tab, start-work, mark-complete, verify-close, reopen, audit and KPI invalidation.
Backend - Documents	multer disk uploads, mimetype allow-list, 10 MB cap, 50 file cap, soft delete, download stream.
Backend - Tasks	Library task lookup, per-JC task rows, custom tasks, completion toggles, hard-delete.
Backend - JR Loose Ends	Edit DRAFT and cancel DRAFT without enum mutation.
Frontend	JobCardDetail orchestrator, 13 tabs, save bar, autosave, mark-complete gate panel, closure form, reopen modal.
Verification	Smoke runner A1..A29 plus setup, 30/30 pass with documented single-engineer skips.

3.2 Delivered file inventory

Area	Files / patches	Purpose
DB	300/301/302/303/304 migrations	Schema expansion and performance indexes.
BE core	jobCards.stateMachine.js, repo, validators, service, controller, routes	Lifecycle and tab-data orchestration.
BE submodules	taskChecklist/* and documents/*	Task and file management.
BE utilities	fileStorage.js	multer disk storage, path safety, upload validation translation.
FE schemas	jobCardSchemas.js	Zod mirror for frontend validation.
FE hooks	useJobCardDetail, useJobCardHistory, useJobCardTasks, useJobCardDocuments, useTaskLibrary, useAutoSave	Data fetching and autosave behavior.
FE page	JobCardDetail.jsx	13-tab orchestration with conditional actions.
FE components	Header, tab bar, save bar, modals, tab components	User experience layer.

LOCKED BUT HONEST

Phase 9 is end-to-end for the main engineer workflow. The maintenance-details and spares-used multi-row CRUD are explicitly deferred to Phase 9 Slice 2 while their database tables already exist.

4. Database Design Logic - How to Think

4.1 Why the database changes are additive

The existing CMCMIS database contains historical Job Cards and legacy operational records. A senior production system does not redesign these tables in place. Phase 9 therefore extends `cmms_jobcard_mst` with nullable MVP fields and creates new child tables for multi-row data. This keeps historical rows valid and gives the new workflow its own safe space.

DATABASE THINKING RULE

If a field is conceptually one value per Job Card, add it as a nullable column. If it can appear multiple times per Job Card, create a child table. If it represents a state transition, write a history row. If it represents an actor, store `employee_id` consistently.

4.2 Ground-truth observations from introspection

Observed fact	Design consequence
19,432 legacy VERIFIED_CLOSED rows existed	All new fields must be nullable; legacy rows become read-only in UI.
Only 4 ASSIGNED MVP Job Cards existed for Phase 9 work queue	Smoke testing can validate lifecycle but multi-engineer checks are limited.
JM_MVP_STATUS already had ASSIGNED, IN_PROGRESS, COMPLETED, VERIFIED_CLOSED, REOPENED	No enum modification required for Job Card lifecycle.
JR_MVP_STATUS did not contain CANCELLED	Use logical cancellation fields instead of enum mutation.
Phase 9 permissions already seeded	No permission-seeding migration needed.
multer not installed initially	Add multer + uuid for document uploads.

4.3 Snake_case decision

D-9.10 - COLUMN STYLE OVERRIDE

Phase 9 uses `snake_case` for 53 new MVP-only columns such as `plug_in_accessories` and `completion_summary`. Existing `JM_*` legacy fields remain untouched. This was chosen because the new fields belong to the MVP workflow, not the original legacy CMMS shape.

4.4 Actor identity decision

D-9.12 - ACTOR IDENTITY

All actor columns use `employee_id VARCHAR(7)`, not `user_id BIGINT`. This keeps JR, JC, `audit_log`, `status_history`, and `cmms_emp_mst` joins aligned.

5. Migration Deep Dive - 300 to 304

Migration	Name	Purpose	Safety logic
300	phase9_jc_columns	ADD 53 snake_case nullable columns to cmms_jobcard_mst for all Phase 9 tabs and closure metadata.	All nullable; legacy rows remain valid.
301	phase9_jc_child_tables	CREATE jc_maintenance_details, jc_spares_used, jc_task_checklist, jc_documents, jc_observations_readings, job_card_status_history.	New tables only; no destructive changes.
302	phase9_task_library	CREATE task_library and seed 45 standard tasks, 15 per category.	Idempotent seed; curated task source.
303	phase9_jr_cancel_columns	ADD JR_CANCELLED_AT, JR_CANCELLED_BY, JR_CANCEL_REASON for logical CANCELLED.	No enum modify.
304	phase9_indexes	ADD idx_jc_completion, idx_jr_cancelled, idx_jc_verified_close_at.	Performance for dashboards, listing, closure queries.

5.1 Migration 300 - 53 Job Card columns

Migration 300 is the largest schema expansion in the phase. It gives the Job Card master table the fields needed for all simple tabs, completion, closure, and reopen metadata. All columns are nullable except customer_acknowledged and reopen_count defaults, so legacy records remain valid.

Column group	Examples	Business reason
Accessories	plug_in_accessories	Record the accessories received with equipment.
Submitted/Received	equipment_submitted_date, submitted_by, received_by	Handover traceability.
Job details	job_type, repair_type, job_request_remarks	Engineer-level classification.
Awaiting/procurement	awaiting_status, supplier_name, po_no, procurement_cost	Track wait and procurement lifecycle.
Contract/warranty	vendor_supplier_name, invoice_no, labour_charges	Vendor-facing repair details.
Observations	observations_text, job_status_display	Engineering observations and display-state axis.
Completion	completion_summary, actual_completion_date, total_hours_spent	Engineer completion evidence.
Closure	reviewed_by, customer_acknowledged, final_closure_notes	LIC verification and customer satisfaction.
Reopen	last_reopened_at, last_reopened_by_employee_id, reopen_count	Track failed closure loops.

5.2 Migration 301 - child tables

Table	Why it exists	Delete policy
jc_maintenance_details	Defect/observation/action rows for future granular maintenance entries.	Hard delete in Slice 2.
jc_spares_used	Multi-row spare parts and cost details.	Hard delete in Slice 2.
jc_task_checklist	Per-JC task instances copied from library or custom tasks.	Hard delete.
jc_documents	File metadata for uploaded evidence.	Soft delete; bytes retained.
jc_observations_readings	Structured observation/value rows.	Append-like operational evidence.
job_card_status_history	Transition history for JC lifecycle.	Never delete.

5.3 Migration 302 - task library

The task library seeds approximately 45 curated tasks, distributed across calibration, inspection, and maintenance categories. The engineer can add library tasks to a Job Card, or write custom tasks for one-off work.

TASK LIBRARY LOGIC

The library is curated reference data. Custom engineer tasks are not promoted into the library automatically. This prevents task-library spam and keeps future admin management clean.

5.4 Migration 303 - logical cancellation

```
DRAFT JR cancel without enum modification:
JR_MVP_STATUS remains DRAFT
JR_CANCELLED_AT is set
JR_CANCELLED_BY is set
JR_CANCEL_REASON may be set
List endpoint hides cancelled drafts by default
```

5.5 Migration 304 - performance indexes

The new indexes support common reads: completion/closure dashboards, hidden cancelled drafts, and closure date lookups. This is small but important because dashboard and inquiry phases rely on predictable query performance.

6. Job Card State Machine

MOST IMPORTANT FILE

The Job Card state machine is the single blessed path for JM_MVP_STATUS changes. Future modules must extend the state machine instead of writing raw UPDATE statements against status columns.

```

ASSIGNED --start-work--> IN_PROGRESS --mark-complete--> COMPLETED --verify-close--> VERIFIED_CLOSED
                                     ^                               |
                                     |                               |
                                     +----- reopen -----+

```

From	Action	To	Allowed by	Critical gates
ASSIGNED	start-work	IN_PROGRESS	Assigned engineer or LIC/SA	Correct assignment and permission.
IN_PROGRESS	patch tab	IN_PROGRESS	Assigned engineer or LIC/SA	Save only; no state transition.
IN_PROGRESS	mark-complete	COMPLETED	Assigned engineer or LIC/SA	Four completion gates.
COMPLETED	verify-close	VERIFIED_CLOSED	LIC/SA	Closure form + customer acknowledgement.
COMPLETED	reopen	IN_PROGRESS	LIC/SA	Reason >= 20 chars.
VERIFIED_CLOSED	reopen	IN_PROGRESS	LIC/SA	Reason + reset closure and completion fields.

6.1 Transition transaction pattern

```

BEGIN TRANSACTION
  SELECT Job Card FOR UPDATE
  validate state-machine transition
  validate ownership and permission
  perform UPDATE on cmms_jobcard_mst
  INSERT job_card_status_history
  INSERT audit_log
COMMIT
  invalidate kpiCache keys

```

NO PARTIAL STATE

If a transition fails, there should be no half-written status, no orphan history row, and no audit row pretending success. Smoke tests verified unchanged state on failed gates.

7. Backend Architecture

7.1 Module structure

Module file / folder	Responsibility
jobCards.stateMachine.js	Canonical transition table and transition validator.
jobCards.validators.js	Zod schemas for tab patching, transitions, task, document, closure, reopen.
jobCards.repo.js	SQL access: detail, history, tab updates, transition writes, audit writes, gate checks.
jobCards.service.js	Business orchestration: ownership, legacy guard, state machine, transactions, gate logic.
jobCards.controller.js	HTTP handlers and response envelope.
jobCards.routes.js	Route middleware, permissions, mounted submodules.
taskChecklist/*	Task checklist sub-feature with library/custom tasks.
documents/*	Multer-backed document upload, download, soft delete.
utils/fileStorage.js	Central disk storage configuration and file path helpers.
lookups/*	Task-library lookup endpoint.
jobRequests/* patches	DRAFT edit/cancel loose ends.

7.2 Service-layer responsibilities

- Load the Job Card under transaction lock for state changes.
- Reject write attempts on legacy read-only Job Cards.
- Check own engineer vs LIC/SA privileges.
- Run state-machine transition validation before modifying status.
- Run pre-completion gates in mark-complete transaction.
- Write audit and status-history rows atomically.
- Invalidate kpiCache after successful transition commits.

7.3 Repository-layer thinking

REPO RULE

The repository knows SQL and column mapping; it does not decide business permissions. The service decides the workflow. This separation makes schema changes manageable and keeps audit logic visible.

7.4 Validator-layer thinking

Zod validators catch shape errors before the service runs. However, validators do not own business truth: the state machine and service still re-check state, permission, and ownership. This prevents client-side UI bugs from mutating production state.

8. API Contract

Method	Endpoint	Permission	Purpose
GET	/api/v1/job-cards/:id	job_card:read-detail	Full JC detail with equipment, parent JR, assignment, flags, data tabs.
GET	/api/v1/job-cards/:id/history	job_card:read-detail	JC status-history timeline.
PATCH	/api/v1/job-cards/:id	job_card:update-tasks	Save tab data; no state transition; no audit due to noise.
POST	/api/v1/job-cards/:id/start-work	job_card:start-work	ASSIGNED -> IN_PROGRESS.
POST	/api/v1/job-cards/:id/mark-complete	job_card:complete	IN_PROGRESS -> COMPLETED after four gates.
POST	/api/v1/job-cards/:id/verify-close	job_card:verify-close	COMPLETED -> VERIFIED_CLOSED; requires customer acknowledgement.
POST	/api/v1/job-cards/:id/reopen	job_card:reopen	COMPLETED/VERIFIED_CLOSED -> IN_PROGRESS with mandatory reason.
GET	/api/v1/job-cards/:id/tasks	job_card:read-detail	List task checklist rows.
POST	/api/v1/job-cards/:id/tasks	job_card:update-tasks	Add library/custom task.
PATCH	/api/v1/job-cards/:id/tasks/taskId	job_card:update-tasks	Toggle task completion.
DELETE	/api/v1/job-cards/:id/tasks/taskId	job_card:update-tasks	Hard-delete task row.
GET	/api/v1/job-cards/:id/documents	job_card:read-detail	List active documents.
POST	/api/v1/job-cards/:id/documents	job_card:update-tasks	Upload document by multer disk storage.
GET	/api/v1/job-cards/:id/documents/docId	job_card:read-detail	Download document stream.
DELETE	/api/v1/job-cards/:id/documents/docId	job_card:update-tasks	Soft-delete document metadata.
PATCH	/api/v1/job-requests/:id	job_request:create/read-own	Edit DRAFT JR loose end.
POST	/api/v1/job-requests/:id/cancel	job_request:create/read-own	Logical cancel for DRAFT JR.
GET	/api/v1/lookups/task-library	authenticated	Task-library lookup filtered by category.

8.1 Why endpoint count grew

Phase 9 intentionally separates lifecycle transitions from tab updates and submodule operations. This avoids large all-purpose endpoints and makes audit expectations obvious. A tab save does not change state. A transition endpoint changes state and writes audit/history. A document endpoint deals with upload/download details. A task endpoint deals with checklist rows.

8.2 Why PATCH tab saves are not audited

D-9.2 AUDIT NOISE DECISION

PATCH /job-cards/:id saves dirty form columns and stamps updated_at/updated_by, but does not write an audit row. Otherwise a 30-second autosave loop would flood audit_log and bury true lifecycle decisions.

9. Frontend Architecture

9.1 Orchestrator page

JobCardDetail.jsx is the orchestration layer. It fetches the card, computes role/ownership flags, decides tab visibility, renders banners, chooses the active tab from URL query parameters, and wires transitions and cache invalidation.

```
/job-cards/:id?tab=observations
-> useJobCardDetail(id)
-> compute isLegacy, isOwnEngineer, isLicSa
-> visibleKeys = base tabs + conditional mark-complete/closure
-> render active tab
-> invalidate detail/history/tasks/documents after mutation
```

9.2 Hooks created

Hook	Purpose	Refresh/cache behavior
useJobCardDetail	Fetch full Job Card detail.	15s polling + focus refresh.
useJobCardHistory	Fetch status timeline.	30s cache.
useJobCardTasks	Fetch checklist rows.	15s cache and refetch on mutation.
useJobCardDocuments	Fetch document metadata.	15s cache and refetch on upload/delete.
useTaskLibrary	Fetch task library by category.	5-minute reference-data cache.
useAutoSave	Debounce and heartbeat tab autosave.	Idle/debounce + 30s heartbeat.

9.3 Components created

Component	Role
DetailHeader	Code, status, equipment summary, assigned engineer, PDF stub.
DetailTabBar	13 conditional tab navigation.
TabSaveBar	Auto-save status, preference toggle, Save as Draft, Save Changes.
SimpleDataTabs	Seven form tabs for text/date/enum fields.
TaskChecklistTab	Library/custom tasks and progress.
DocumentsTab	Upload/list/delete/download documents.
MarkCompleteTab	Gate panel and completion form.
ClosureTab	Quality review + customer acknowledgement.
ReopenModal	Mandatory reason reopen modal.

FRONTEND PRINCIPLE

The FE makes the right action easy, but never trusts itself. Buttons are hidden or disabled when not allowed, but the backend remains the real authority.

10. Thirteen Tab UX

#	Tab	Purpose
1	Plug In / Accessories	Free-text accessories captured with the instrument.
2	Submitted & Received	Submitted date/by, received date/by, handover trace.
3	Job Card Details	Engineer-level job type, repair type, planned completion, remarks.
4	Maintenance Details	Prepared table for defect/action rows; delivery note: multi-row CRUD is Slice 2 stub.
5	Equipments Used Details	Free-text instruments/tools used while executing the job.
6	Awaiting Information	Waiting state, supplier, dates, procurement references.
7	Spares Used Details	Prepared table for spares rows; delivery note: multi-row CRUD is Slice 2 stub.
8	Contract / Warranty	Vendor/supplier, gate pass, invoice, charges, warranty execution details.
9	Observations	Free-text observations plus structured observation readings.
10	Task Checklist	Library + custom tasks, progress, toggle completion, hard delete rows.
11	Documents	Multipart upload, soft delete, document type gates.
12	Mark as Complete	Gate panel and completion summary.
13	Closure	Quality review + customer acknowledgement + final closure notes.

10.1 Conditional tab logic

The first eleven tabs are base tabs. Mark Complete appears only when the card is IN_PROGRESS and the actor can complete. Closure appears only when the card is COMPLETED and the actor is LIC/SA with verify-close permission. This keeps the page from showing impossible actions.

10.2 Legacy read-only behavior

D-9.14 LEGACY BANNER

A Job Card is treated as legacy when it has no parent_jr_no, no assigned_engineer_employee_id, and status VERIFIED_CLOSED. The page renders a read-only banner, hides writes, and the backend returns 409 for write attempts.

10.3 Tab save behavior

```
User edits simple tab
-> form becomes dirty
-> auto-save waits for idle/debounce or 30s heartbeat
-> PATCH /job-cards/:id with dirty fields only
-> status stays IN_PROGRESS
-> no audit row
-> updated_at/updated_by stamped
```

11. Engineer Execution Flow

11.1 Start Work

When a Job Card is ASSIGNED, the assigned engineer or LIC/SA can start work. Start Work transitions the card into IN_PROGRESS and makes the data tabs editable. This is the entry point into the engineer workspace.

11.2 Working state

During IN_PROGRESS, the engineer fills the tabs, records observations, adds tasks, uploads documents, and prepares evidence. The system supports long-lived work through manual save and auto-save.

11.3 Completion handoff

The engineer cannot mark the card complete unless the server verifies the four gates. Once complete, the card becomes read-only to the engineer and waits for LIC/SA verification.

```
ASSIGNED
  engineer clicks Start Work
IN_PROGRESS
  save tabs + tasks + documents + observations
  mark-complete runs gates
COMPLETED
  LIC verifies and closes OR reopens
VERIFIED_CLOSED / IN_PROGRESS
```

ENGINEER MENTAL MODEL

The page behaves like a live workbench. The engineer should not think about database state; they should see clear tabs, progress gates, evidence upload, and one obvious completion path.

12. Pre-completion Gates

D-9.4 GATE RULE

Mark Complete is not just a button. It is a server-verified readiness check. If any gate fails, the card remains IN_PROGRESS and no audit/history row is written.

Gate	Required condition	Why it exists
Task gate	All required tasks are completed.	Prevents empty checklist closure.
Observation gate	Structured observation rows exist OR observations_text has sufficient detail.	Ensures technical evidence exists.
Calibration certificate gate	CALIBRATION workflows need CALIBRATION_CERT document.	Calibration closure requires certificate evidence.
Required document gate	Required docs uploaded where applicable.	Evidence package completeness.

12.1 Failure behavior

```
mark-complete WHEN gates FAIL:
-> 400 PRECOMPLETION_GATES_FAILED
-> response contains details.gates[]
-> JC status unchanged
-> no audit row
-> no status history row
```

12.2 Success behavior

```
mark-complete WHEN all gates GREEN:
-> JM_MVP_STATUS: IN_PROGRESS -> COMPLETED
-> JM_JobEndDate stamped
-> completion_summary + actual_completion_date + total_hours_spent persisted
-> audit_log: JC_MARK_COMPLETE
-> job_card_status_history row
-> kpiCache invalidated
```

13. Closure and Reopen

13.1 Verify Close

Verify Close belongs to Lab In-Charge/Super Admin. It represents acceptance of the completed engineering work. The closure form includes review details, customer handover details, a hard customer acknowledgement checkbox, and final notes.

Closure section	Fields	Rule
Quality Review	reviewed_by, review_date, review_comments	Reviewer and comments required.
Customer Acknowledgement	equipment_received_by_customer, customer_received_date, customer_acknowledged	customer_acknowledged must be true.
Final Closure	final_closure_notes, verified_closed_by_employee_id, verified_closed_at	Stamped on verify-close.

Q-9 HARD RULE

If customer_acknowledged is false, the card must not close. The correct operational action is to reopen and fix the issue.

13.2 Reopen

Reopen sends a completed or verified-closed card back into IN_PROGRESS. It requires a reason, increments reopen_count, stamps last_reopened_at/by, clears completion and closure fields where applicable, and preserves data tabs.

```
reopen FROM VERIFIED_CLOSED:
-> status becomes IN_PROGRESS
-> completion fields cleared
-> closure fields cleared
-> reopen_count increments
-> observations, task data, documents, and other tabs preserved
-> audit + status_history row with reason
```

WHY PRESERVE DATA

Reopen means the work needs correction, not that previous evidence should be erased. Preserving observations and documents protects auditability and engineer context.

14. Documents and File Storage

14.1 Multer disk storage

Documents are stored on local disk under storage/job-cards/<section_job_no>/<uuid>.<ext>. Metadata is stored in jc_documents. UUID filenames defeat path traversal and avoid original filename collisions.

Rule	Value	Reason
Max file size	10 MB	Protect server storage and request time.
Max files per JC	50	Avoid unbounded evidence packages.
Soft warning	40 files	User can prepare before hitting hard cap.
Allowed mimetypes	PDF, JPEG, PNG, DOCX, XLSX	Expected engineering evidence formats.
Delete policy	Soft delete	Keep file bytes and metadata for audit.
Future storage	S3/R2 compatible swap	fileStorage.js centralizes storage logic.

14.2 Upload path

```
POST /job-cards/:id/documents
-> multer validates size and mimetype
-> diskStorage writes file
-> service validates JC, legacy flag, and write access
-> insert jc_documents metadata row
-> audit_log: JC_DOC_UPLOAD
```

14.3 Delete path

```
DELETE /job-cards/:id/documents/:docId
-> uploader OR LIC/SA required
-> jc_documents.deleted_at set
-> deleted_by_employee_id stamped
-> audit_log: JC_DOC_DELETE
-> bytes retained on disk
```

15. Task Checklist

15.1 Library + custom model

The task checklist is a per-Job-Card set of work items. A task can be copied from task_library or created as a custom task. Library task text is snapshotted into jc_task_checklist so future library edits do not rewrite historical checklists.

Path	Database effect	Why
Add library task	task_id set, is_custom false, task_text copied from task_library.	Reusable standard procedure.
Add custom task	task_id null, is_custom true, task_text from engineer.	One-off operational flexibility.
Toggle task	is_completed, completed_by_employee_id, completed_at updated.	Completion evidence.
Delete task	Hard delete task row.	Task rows are draft-like workbench items.

15.2 Library categories

Category	Used by	Examples
CALIBRATION	Calibration workflows	Pre-calibration verification, reference standard connection, certificate generation.
INSPECTION	Inspection/repair workflows	Visual inspection, functional test, diagnostic report.
MAINTENANCE	Maintenance/repair flows	Fault identification, component replacement, burn-in test.

TASK DESIGN PRINCIPLE

A checklist is not just UI. It is a completion gate, an audit proof, and a way to standardize technical procedure without blocking custom engineer judgement.

16. Observations and Readings

16.1 Observations text

The Observations tab captures technical notes, fault descriptions, measured behavior, and engineering findings. It is the minimal fallback evidence required for mark-complete when structured observation rows are not used.

16.2 Structured readings

The jc_observations_readings table supports parameter/value/unit rows with reading_type. This is future-friendly for calibration and inspection reports, even if the initial workflow also accepts free-text observations.

Field	Purpose
parameter	What was measured or inspected.
value	Measured value or qualitative result.
unit	Measurement unit.
reading_type	PRE_CAL, POST_CAL, INSPECTION, OTHER.
notes	Context, tolerance notes, defects, or special comments.
recorded_by_employee_id	Engineer identity for accountability.

16.3 Two-axis state

Q-8 DECISION

job_status_display is an engineer-facing label such as AWAITING_FOR_VENDOR or HOLD. It is separate from system status JM_MVP_STATUS. This allows the card to be system IN_PROGRESS while operationally waiting on vendor.

17. JR Loose Ends - Edit and Cancel Draft

17.1 Edit DRAFT

Phase 9 also closes Job Request loose ends. Users can edit their own DRAFT Job Requests. Non-DRAFT requests remain locked because the workflow has already moved into approval or execution.

17.2 Cancel DRAFT

Cancel is logical because the JR enum does not include CANCELLED. The system sets cancellation metadata and hides the row from normal lists instead of modifying the enum or deleting the record.

```
DRAFT JR cancellation:
  update JR_CANCELLED_AT = NOW(6)
  update JR_CANCELLED_BY = actor.employee_id
  update JR_CANCEL_REASON = optional reason
  keep JR_MVP_STATUS = DRAFT
  list endpoint excludes cancelled drafts by default
```

WHY NOT DELETE

A cancelled request still matters historically: who cancelled, when, and why. Logical cancellation preserves audit and avoids referential surprises.

18. RBAC and Row-Level Scope

18.1 Defense-in-depth matrix

Layer	Gate
Frontend	Hide/disable buttons when missing permission, wrong status, legacy row, or not own/LIC-SA.
Route middleware	authenticate + authorize(permission).
Row-level scope	job_card scope identifies owner/assigned engineer context.
Service	Re-check ownership and role inside transaction.
State machine	Reject illegal state/action and missing permission.

18.2 Role behavior

Role	Read detail	Write tabs	Transitions
Normal User	No general JC write; can see relevant linked context where permitted.	No	No
View Only	Yes for visibility purposes.	No	No
Lab Engineer	Assigned cards; off-assignment read-only behavior.	Own assigned IN_PROGRESS cards.	Start and complete own cards.
Lab In-Charge	All operational cards.	Can assist/override where permitted.	Verify-close and reopen.
Super Admin	All cards.	Can assist/override.	Verify-close and reopen.

18.3 Legacy write protection

Legacy rows are protected at the frontend and service layers. The UI shows a read-only banner, and any write attempt returns a conflict response. This avoids corrupting the historical CMMS import.

19. Smoke Verification

19.1 Verification summary

GREEN RESULT

Smoke result: 30/30 checks passed. A1..A29 plus setup were green. Three cross-engineer checks were skipped because the database had only one active LAB_ENGINEER; their design intent was verified by code review.

Test	Scenario	Expected result
A1	GET JC detail	200 with full detail payload.
A2	Off-assignment read	200 read-only or scoped response based on role.
A3	Start work valid	ASSIGNED -> IN_PROGRESS + audit + history.
A4	Start work wrong state	409 / illegal transition.
A5	Patch tab save	Dirty fields saved, status preserved, no noisy audit.
A6	Task library fetch	45 seeded tasks accessible by category.
A7	Cross-engineer write	Skipped in one-engineer environment; code-review verified.
A8	Upload document	Multer stores file and metadata row.
A9	Download document	Authenticated stream response.
A10	Soft-delete document	deleted_at set; bytes retained.
A11	Delete doc as non-uploader	Skipped in one-engineer environment; design verified.
A12	Tab PATCH dirty-cols only	Only selected columns touched; update stamps applied.
A13	Mark-complete missing gates	400 PRECOMPLETION_GATES_FAILED with gate details.
A14	Mark-complete all green	IN_PROGRESS -> COMPLETED + summary + audit + history.
A15	Verify-close customer acknowledgement	Hard-rule schema/service gate.
A16	Verify-close success	COMPLETED -> VERIFIED_CLOSED + closure persisted.
A17	Reopen short reason	422 validation error.
A18	Reopen success	Back to IN_PROGRESS, resets completion/closure, preserves data tabs.
A19	JR edit DRAFT	PATCH DRAFT allowed; non-DRAFT blocked.
A20	JR cancel DRAFT	Logical cancellation columns set; list hides row.
A29	Legacy JC read-only	GET 200 is_legacy=true; PATCH write returns 409.

19.2 Atomic guarantees proven

Operation	Guarantee
Failed mark-complete	State unchanged, no audit row, no history row.
Successful mark-complete	COMPLETED state, completion fields, audit/history, cache invalidation.
Reopen verified-closed	Status returns IN_PROGRESS, completion/closure reset, data tabs preserved.
PATCH tab save	Only dirty columns touched, no status change, no noisy audit.
Legacy write attempt	409 conflict; no mutation.

19.3 Known test limitation

SINGLE-ENGINEER LIMITATION

The smoke environment had one active Lab Engineer: TE00225. Cross-engineer scenarios need a second seeded engineer to fully exercise off-assignment write denial behavior in runtime.

20. Honest Scope Notes

20.1 Maintenance and Spares CRUD

DEFERRED TO PHASE 9 SLICE 2

Maintenance Details and Spares Used multi-row tables and indexes are migrated, but their full per-row CRUD UI/API are Slice 2 stubs. Engineers can still complete workflows using Observations and Contract/Warranty fields; mark-complete accepts structured readings or free-text observation fallback.

20.2 PDF and calibration certificate

PDF generation and calibration certificate output remain Phase 11. The verify-close service has a future hook for equipment.last_cal_date update, but it is intentionally stubbed until certificate generation ships.

20.3 Notifications and co-editing

Email notifications, SSE co-editing warnings, and cloud storage are deferred. The current file storage is centralized so moving from local disk to S3/R2 later is a service-utility swap rather than a rewrite.

Deferred item	Target	Reason
Maintenance multi-row CRUD	Phase 9 Slice 2	Tables are ready; needs dedicated UI and API.
Spares multi-row CRUD	Phase 9 Slice 2	Similar table-driven UI pattern.
Equipment re-assignment mid-job	Phase 10	Needs workload and audit design.
PDF report and certificate	Phase 11	Requires layout and certificate governance.
Email notifications	Phase 12	Requires template and delivery policy.
Cloud storage	Slice 2/future	Local disk is adequate for current scale.

21. How to Think for Backend Logic Development

21.1 Senior backend thinking sequence

1. Start from the database reality, not from UI imagination. Inspect the table, enum, index, and row counts first.
2. Separate state changes from data saves. A tab PATCH is not a transition. A transition writes audit/history.
3. Lock the row for transition operations. Use transaction boundaries to avoid half-state.
4. Run the state machine before the UPDATE. Never UPDATE status directly.
5. Re-check ownership inside the service even if the route is authorized.
6. Write audit rows only for events that matter operationally. Autosave is too noisy.
7. Invalidate KPI/cache after successful commit, not before.
8. Preserve legacy data by making new columns nullable and legacy rows read-only if needed.

21.2 Database design checklist

Question	Answer pattern
One value per JC?	Column on cmms_jobcard_mst, nullable if legacy table.
Multiple rows per JC?	New child table with jc_section_no FK.
State change?	State machine + history + audit.
Actor field?	employee_id VARCHAR(7).
Historical row?	Do not mutate destructively; render read-only when required.
Long-running form?	PATCH dirty fields + autosave, not transition.
Evidence file?	Document metadata row + fileStorage utility.

21.3 Controller-service-repo discipline

Controller: parse request, pass actor and body to service
 Service: decide business logic, transactions, ownership, state machine
 Repo: SQL only, no policy decisions
 Validator: shape-level safety, not business authority
 State machine: final transition authority

22. Frontend Logic Development Notes

22.1 UI should mirror, not replace backend rules

The frontend computes action availability to improve user experience. However, each action still goes through backend permission, ownership, legacy, and state-machine checks.

22.2 Autosave logic

AUTO-SAVE BEHAVIOR

Auto-save is on by default. The save bar shows status such as Saving, Saved, Error, or Unsaved changes. The user can still manually save. Dirty field PATCHes avoid overwriting unrelated tab fields.

22.3 Conditional tab UX

Tabs should not expose impossible operations. Mark Complete appears only when completion can be attempted. Closure appears only when LIC/SA is reviewing a COMPLETED Job Card. Legacy tabs render read-only.

22.4 Error state handling

Error	UI response
PRECOMPLETION_GATES_FAILED	Show gate panel with red gates.
409 legacy write	Show legacy read-only message.
403 permission	Hide buttons; show read-only state if opened.
422 validation	Field-level error under the relevant input.
Upload too large/mimetype rejected	Document tab error banner.

23. Future Handoff

23.1 Immediate next phase

NEXT RECOMMENDED WORK

Phase 9 Slice 2: finish Maintenance Details and Spares Used multi-row CRUD. The database foundation is already present, so this is mostly BE submodule + FE table form work.

23.2 Phase 10-12 route

Phase	Focus	Why it comes next
Phase 9 Slice 2	Maintenance + Spares CRUD	Completes the two currently stubbed multi-row tabs.
Phase 10	Equipment master CRUD + engineer re-assignment mid-job	Strengthens operational control and workload management.
Phase 11	PDF report + calibration certificate + equipment last_cal_date update	Converts operational data into official output artifacts.
Phase 12	Notifications + time-clock + cloud storage	Improves enterprise readiness and operational communication.

23.3 Handoff rules for future developers

- Do not bypass `jobCards.stateMachine.js` for status changes.
- Do not allow writes to legacy Job Cards.
- Do not audit autosave-like noisy updates.
- Do not modify enums unless a future locked decision explicitly approves it.
- Use `employee_id` for new actor columns unless the architecture changes globally.
- Keep document storage behind `utils/fileStorage.js` so cloud migration remains easy.

24. Appendix A - Phase 9 Field Map by Tab

Tab / area	Columns
Plug In / Accessories	plug_in_accessories
Submitted & Received	equipment_submitted_date, submitted_by, equipment_received_date_actual, received_by
Job Card Details	instrument_received_date, job_complete_planned_date, job_type, repair_type, job_request_remarks
Equipments Used	equipments_used
Awaiting Information	awaiting_for, awaiting_status, supplier_name, awaiting_from_date, awaiting_clear_date, attended_by, indent_no, indent_date, mirv_no, mirv_date, po_no, po_date, procurement_cost
Contract / Warranty	vendor_supplier_name, intimation_sent_on, sent_to_vendor_date, received_from_vendor_date, gate_pass_no, gate_pass_issued_date, cost_of_component, labour_charges, invoice_no, invoice_recd_on
Observations	observations_text, job_status_display
Completion	completion_summary, actual_completion_date, total_hours_spent, marked_complete_by_employee_id, marked_complete_at
Closure	reviewed_by, review_date, review_comments, equipment_received_by_customer, customer_received_date, customer_acknowledged, final_closure_notes, verified_closed_by_employee_id, verified_closed_at
Reopen	last_reopened_at, last_reopened_by_employee_id, reopen_count

25. Appendix B - Decision Traceability Matrix

Decision	Short name	Impact
D-9.1	One JC detail page with 13 tabs	Deep-linkable /job-cards/id?tab=... page; tabs are conditional by status and permission.
D-9.2	Auto-save 30s + manual save	Long engineer forms need draft safety; tab PATCH preserves state.
D-9.3	Child tables for multi-row data	Task checklist, documents, observation readings, maintenance and spares use separate child tables.
D-9.4	Four pre-completion gates	Tasks done, observations recorded, calibration cert for calibration workflows, required docs uploaded.
D-9.5	Closure requires customer acknowledgement	Verify-close only succeeds when customer_acknowledged is true.
D-9.6	Reopen preserves work data	Completion/closure fields reset, but observations and other data tabs remain.
D-9.7	Task library category-filtered	Task picker filters by workflow category with a show-all escape hatch.
D-9.8	Documents on local disk via multer	10 MB/file, 50 files/JC, mimetype allow-list, soft delete.
D-9.9	15s detail refresh	Active engineer pages need tighter feedback than list pages.
D-9.10	snake_case new Phase 9 columns	53 MVP-only columns use canonical snake_case rather than JM_* prefix.
D-9.11	CANCELLED is logical only	Avoid enum modify; use cancellation columns and list filters.
D-9.12	Actor identity = employee_id	All by-actor columns use employee_id VARCHAR(7), consistent with audit and legacy data.
D-9.13	JR reopen remains out of scope	Rejected JR reopen is not part of Phase 9.
D-9.14	Legacy JCs are read-only	19,432 historical VERIFIED_CLOSED rows render with read-only banner and block writes.

26. Final Seal

PHASE 9 SEALED

Phase 9 is documented as delivered end-to-end for Job Card Detail + Observations + Engineer Execution. The system now supports the operational path from assigned Job Card to engineer work, evidence capture, completion, closure, reopen, and audit trail.

DS closing note: this phase is the operational heart of CMCMIS. The next engineering win is to complete the two Slice 2 stubs and then move into official report/certificate generation.

Appendix C.1 - Tab Deep Brief: Plug In / Accessories

TAB PURPOSE

Free-text accessories captured with the instrument.

User thinking

The Plug In / Accessories tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Plug In / Accessories contributes to execution context. Normal tab saves are intentionally not audit-log events to avoid autosave noise.

Appendix C.2 - Tab Deep Brief: Submitted & Received

TAB PURPOSE

Submitted date/by, received date/by, handover trace.

User thinking

The Submitted & Received tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Submitted & Received contributes to execution context. Normal tab saves are intentionally not audit-log events to avoid autosave noise.

Appendix C.3 - Tab Deep Brief: Job Card Details

TAB PURPOSE

Engineer-level job type, repair type, planned completion, remarks.

User thinking

The Job Card Details tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Job Card Details contributes to execution context. Normal tab saves are intentionally not audit-log events to avoid autosave noise.

Appendix C.4 - Tab Deep Brief: Maintenance Details

TAB PURPOSE

Prepared table for defect/action rows; delivery note: multi-row CRUD is Slice 2 stub.

User thinking

The Maintenance Details tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Maintenance Details contributes to execution context. Normal tab saves are intentionally not audit-log events to avoid autosave noise.

Appendix C.5 - Tab Deep Brief: Equipments Used Details

TAB PURPOSE

Free-text instruments/tools used while executing the job.

User thinking

The Equipments Used Details tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Equipments Used Details contributes to execution context. Normal tab saves are intentionally not audit-log events to avoid autosave noise.

Appendix C.6 - Tab Deep Brief: Awaiting Information

TAB PURPOSE

Waiting state, supplier, dates, procurement references.

User thinking

The Awaiting Information tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Awaiting Information contributes to execution context. Normal tab saves are intentionally not audit-log events to avoid autosave noise.

Appendix C.7 - Tab Deep Brief: Spares Used Details

TAB PURPOSE

Prepared table for spares rows; delivery note: multi-row CRUD is Slice 2 stub.

User thinking

The Spares Used Details tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Spares Used Details contributes to execution context. Normal tab saves are intentionally not audit-log events to avoid autosave noise.

Appendix C.8 - Tab Deep Brief: Contract / Warranty

TAB PURPOSE

Vendor/supplier, gate pass, invoice, charges, warranty execution details.

User thinking

The Contract / Warranty tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Contract / Warranty contributes to execution context. Normal tab saves are intentionally not audit-log events to avoid autosave noise.

Appendix C.9 - Tab Deep Brief: Observations

TAB PURPOSE

Free-text observations plus structured observation readings.

User thinking

The Observations tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Observations contributes to execution context. Normal tab saves are intentionally not audit-log events to avoid autosave noise.

Appendix C.10 - Tab Deep Brief: Task Checklist

TAB PURPOSE

Library + custom tasks, progress, toggle completion, hard delete rows.

User thinking

The Task Checklist tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Task Checklist has direct evidence or lifecycle impact. Its operations either create auditable evidence rows or participate in transition gates.

Appendix C.11 - Tab Deep Brief: Documents

TAB PURPOSE

Multipart upload, soft delete, document type gates.

User thinking

The Documents tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Documents has direct evidence or lifecycle impact. Its operations either create auditable evidence rows or participate in transition gates.

Appendix C.12 - Tab Deep Brief: Mark as Complete

TAB PURPOSE

Gate panel and completion summary.

User thinking

The Mark as Complete tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Mark as Complete has direct evidence or lifecycle impact. Its operations either create auditable evidence rows or participate in transition gates.

Appendix C.13 - Tab Deep Brief: Closure

TAB PURPOSE

Quality review + customer acknowledgement + final closure notes.

User thinking

The Closure tab exists so the engineer or reviewer can capture one focused part of the Job Card evidence without mixing it with unrelated data. The tab separates mental load and makes partial save/autosave safe.

Backend thinking

- Validate shape at route boundary.
- Keep tab save as PATCH data update, not lifecycle transition.
- Check write access: assigned engineer or LIC/SA, non-legacy, IN_PROGRESS where applicable.
- Touch only dirty columns to reduce accidental overwrite risk.
- Stamp updated_at and updated_by consistently.

Frontend thinking

- Keep fields grouped and readable.
- Show save state clearly through TabSaveBar.
- Disable form on legacy/read-only/incorrect status.
- Make errors local to the field where possible.
- Do not hide data just because the user cannot edit it.

Audit and data integrity

AUDIT IMPACT

Closure has direct evidence or lifecycle impact. Its operations either create auditable evidence rows or participate in transition gates.

Appendix D - Endpoint Deep Briefs

WHY THIS APPENDIX EXISTS

Each endpoint has a separate design reason. This appendix is intentionally verbose so a future developer can implement, debug, extend, or test Phase 9 without rereading raw transcripts.

Appendix D.1 - Endpoint Brief: GET /api/v1/job-cards/:id

ENDPOINT PURPOSE

Full JC detail: Reads canonical detail, flags legacy rows, provides all tab data.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint supports read-side orchestration. It should return canonical field names that the frontend can consume without remembering legacy column names.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for GET /api/v1/job-cards/:id

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.2 - Endpoint Brief: GET /api/v1/job-cards/:id/history

ENDPOINT PURPOSE

JC history: Feeds the timeline and reopen banner.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint supports read-side orchestration. It should return canonical field names that the frontend can consume without remembering legacy column names.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for GET /api/v1/job-cards/:id/history

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.3 - Endpoint Brief: PATCH /api/v1/job-cards/:id

ENDPOINT PURPOSE

Save tab data: Dirty field persistence with no status transition.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint is a data save, not a lifecycle transition. It should touch only the fields included in the validated payload, preserve current state, and stamp updated metadata. It should not write a lifecycle audit row.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for PATCH /api/v1/job-cards/:id

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.4 - Endpoint Brief: POST /api/v1/job-cards/:id/start-work

ENDPOINT PURPOSE

Start work: ASSIGNED to IN_PROGRESS.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint is a lifecycle or terminal action. It must be transaction-wrapped, must invoke the relevant state-machine path, must write history/audit rows on success, and must roll back on failure.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for POST /api/v1/job-cards/:id/start-work

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.5 - Endpoint Brief: POST /api/v1/job-cards/:id/mark-complete

ENDPOINT PURPOSE

Mark complete: Gate-checked IN_PROGRESS to COMPLETED.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint is a lifecycle or terminal action. It must be transaction-wrapped, must invoke the relevant state-machine path, must write history/audit rows on success, and must roll back on failure.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for POST /api/v1/job-cards/:id/mark-complete

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.6 - Endpoint Brief: POST /api/v1/job-cards/:id/verify-close

ENDPOINT PURPOSE

Verify close: LIC/SA closure with customer acknowledgement.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint is a lifecycle or terminal action. It must be transaction-wrapped, must invoke the relevant state-machine path, must write history/audit rows on success, and must roll back on failure.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for POST /api/v1/job-cards/:id/verify-close

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.7 - Endpoint Brief: POST /api/v1/job-cards/:id/reopen

ENDPOINT PURPOSE

Reopen: Reason-based reopen from COMPLETED or VERIFIED_CLOSED.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint is a lifecycle or terminal action. It must be transaction-wrapped, must invoke the relevant state-machine path, must write history/audit rows on success, and must roll back on failure.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for POST /api/v1/job-cards/:id/reopen

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.8 - Endpoint Brief: GET /api/v1/job-cards/:id/tasks

ENDPOINT PURPOSE

List tasks: Task checklist tab.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint supports the checklist submodule. Checklist rows are execution evidence and also participate in completion gates.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for GET /api/v1/job-cards/:id/tasks

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.9 - Endpoint Brief: POST /api/v1/job-cards/:id/tasks

ENDPOINT PURPOSE

Add task: Library/custom task creation.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint supports the checklist submodule. Checklist rows are execution evidence and also participate in completion gates.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for POST /api/v1/job-cards/:id/tasks

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.10 - Endpoint Brief: PATCH /api/v1/job-cards/:id/tasks/:taskId

ENDPOINT PURPOSE

Toggle task: Optimistic completion toggles.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint is a data save, not a lifecycle transition. It should touch only the fields included in the validated payload, preserve current state, and stamp updated metadata. It should not write a lifecycle audit row.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for PATCH /api/v1/job-cards/:id/tasks/:taskId

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.11 - Endpoint Brief: DELETE /api/v1/job-cards/:id/tasks/:taskId

ENDPOINT PURPOSE

Delete task: Hard delete task row.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint supports the checklist submodule. Checklist rows are execution evidence and also participate in completion gates.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for DELETE /api/v1/job-cards/:id/tasks/:taskId

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.12 - Endpoint Brief: GET /api/v1/job-cards/:id/documents

ENDPOINT PURPOSE

List docs: Documents tab metadata.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint belongs to the document evidence submodule. It must protect path safety, enforce file rules, keep document metadata queryable, and avoid exposing raw disk paths to the frontend.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for GET /api/v1/job-cards/:id/documents

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.13 - Endpoint Brief: POST /api/v1/job-cards/:id/documents

ENDPOINT PURPOSE

Upload doc: Multer disk upload and metadata insert.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint belongs to the document evidence submodule. It must protect path safety, enforce file rules, keep document metadata queryable, and avoid exposing raw disk paths to the frontend.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for POST /api/v1/job-cards/:id/documents

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.14 - Endpoint Brief: GET /api/v1/job-cards/:id/documents/:docId

ENDPOINT PURPOSE

Download doc: Authenticated file stream.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint belongs to the document evidence submodule. It must protect path safety, enforce file rules, keep document metadata queryable, and avoid exposing raw disk paths to the frontend.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for GET /api/v1/job-cards/:id/documents/:docId

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.15 - Endpoint Brief: DELETE /api/v1/job-cards/:id/documents/:docId

ENDPOINT PURPOSE

Delete doc: Soft-delete metadata and preserve bytes.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint belongs to the document evidence submodule. It must protect path safety, enforce file rules, keep document metadata queryable, and avoid exposing raw disk paths to the frontend.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for DELETE /api/v1/job-cards/:id/documents/:docId

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.16 - Endpoint Brief: PATCH /api/v1/job-requests/:id

ENDPOINT PURPOSE

Edit draft JR: DRAFT-only loose end.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint is a data save, not a lifecycle transition. It should touch only the fields included in the validated payload, preserve current state, and stamp updated metadata. It should not write a lifecycle audit row.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for PATCH /api/v1/job-requests/:id

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.17 - Endpoint Brief: POST /api/v1/job-requests/:id/cancel

ENDPOINT PURPOSE

Cancel draft JR: Logical cancellation without enum modify.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint is a lifecycle or terminal action. It must be transaction-wrapped, must invoke the relevant state-machine path, must write history/audit rows on success, and must roll back on failure.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for POST /api/v1/job-requests/:id/cancel

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix D.18 - Endpoint Brief: GET /api/v1/lookups/task-library

ENDPOINT PURPOSE

Task library: Reference task lookup filtered by category.

Request logic

The request enters through authenticate middleware first. The route then applies permission-level authorization. For write operations, the service layer re-checks row ownership, legacy protection, and valid lifecycle state before any mutation occurs.

Service logic

This endpoint supports the checklist submodule. Checklist rows are execution evidence and also participate in completion gates.

Failure behavior

Failure type	Expected behavior
Unauthenticated	401 from authenticate middleware.
Missing permission	403 from route/service authorization.
Wrong status	409 illegal transition or conflict.
Invalid payload	422 validation error with field details.
Legacy write attempt	409 read-only legacy conflict.

Testing note

Smoke checklist for GET /api/v1/lookups/task-library

1. Valid actor succeeds where appropriate.
2. Wrong role is blocked.
3. Wrong state is blocked.
4. Legacy rows are read-only.
5. Audit/history expectations match action type.

Appendix E - Final Developer Checklist

Checklist item	Status
Migrations are additive and idempotent	Locked
Job Card status changes route through state machine	Locked
Transitions write audit + status history rows	Locked
Legacy Job Cards are read-only	Locked
Autosave avoids audit spam	Locked
Documents use central fileStorage utility	Locked
Task library is curated; custom tasks remain per-card	Locked
Mark-complete gates are server-side	Locked
Customer acknowledgement is required to close	Locked
Phase 9 Slice 2 remains for Maintenance/Spares CRUD	Known next step

END OF PHASE 9 DOCUMENT

This document is now seal-packed for Phase 9 Job Card Detail + Observations + Engineer execution.